Apache Avro

What is Avro?

Apache Avro is a **data serialization system** designed for:

- **Compact** binary serialization
- **Fast** encoding/decoding
- **Schema-based** data exchange
- **Interoperability** across languages

Used heavily in **Kafka**, **event-driven architectures**, and **big-data pipelines** (Hadoop, Spark).

When to Use Avro

- High-throughput messaging
- Event-driven microservices
- Schema-controlled data interchange
- Storage formats (Parquet uses Avro internally)

Not ideal for

- Human-readable APIs → JSON/REST is better
- Browsers (no direct gRPC support)
- Very complex typed RPC → GraphQL preferred

Schema Structure

Avro schemas are written in **JSON** and define `fields` (name + type).

- **Primitive Types** `null`, `boolean`, `int`, `long`, `float`, `double`, `bytes`, `string`
- **Complex Types** `record`, `enum`, `array`, `map`, `fixed`, `union`
- optional defaults

```
{
  "type": "record",
  "name": "User",
  "fields": [
    { "name": "id", "type": "string" },
    { "name": "age", "type": "int" },
    { "name": "isActive", "type": "boolean", "default": true }
  ]
}
```

How Avro Serializes

1. **Writer schema** is used to encode the data.
2. **Reader schema** is used to decode the data.

- No field names included in the binary (unlike JSON).Only the raw values go into the payload.
- Uses **variable-length encoding** for integers (zig-zag + varint).

Compatibility Types

If schemas differ, Avro applies **schema resolution rules**.

Old Schema

```
{
  "type": "record",
  "name": "User",
  "fields": [
    { "name": "name", "type": "string" }
  ]
}
```

New Reader Schema (Backward Compatible)

```
{
  "type": "record",
  "name": "User",
  "fields": [
    { "name": "name", "type": "string" },
    { "name": "age", "type": ["null", "int"], "default": null }
  ]
}
```

Avro in Java (Spring Boot)

```
Schema schema = new Schema.Parser().parse(Resources.getResource("user.avsc"));
GenericRecord user = new GenericData.Record(schema);
user.put("id", "abc123");
user.put("age", 30);

ByteArrayOutputStream out = new ByteArrayOutputStream();
BinaryEncoder encoder = EncoderFactory.get().binaryEncoder(out, null);
DatumWriter<GenericRecord> writer = new GenericDatumWriter<>(schema);
writer.write(user, encoder);
encoder.flush();
```

```
DatumReader<GenericRecord> reader = new GenericDatumReader<>(schema);
BinaryDecoder decoder = DecoderFactory.get().binaryDecoder(bytes, null);
GenericRecord user = reader.read(null, decoder);
```

Why Avro is popular in Kafka

- Messages are tiny → higher throughput
- Strong schema guarantees for producers/consumers
- Integrates with **Schema Registry** (Confluent, Apicurio, Redpanda)

Typical Pattern

- Producer writes message using **writer schema**
- Schema is registered in **Schema Registry**
- Consumer fetches schema ID + binary payload
- Registry provides the **reader schema**

```
[ magic byte | schema id | avro binary ]
```

Resources